CSC311 Introduction to Machine Learning
Project Instruction (Fall 2026)*

Alice Gao, Marina Tawfik

Last Updated: September 16, 2026

Contents

1 Introduction **2**

2 How We Used AI to Write This Document **2**

3 Components and Grading Scheme **3**
- 3.1 Data Collection . . . 3
- 3.2 Team Formation . . . 3
- 3.3 Project Proposal . . . 3
- 3.4 Model Prediction . . . 4
- 3.5 Report . . . 5

4 How to Choose Your Teammates **6**

5 How to Work on the Project **7**

*These instructions are adapted from the work of Professor Lisa Zhang at the University of Toronto, Mississauga.

1 Introduction

Welcome to the CSC311 Machine Learning Project. The project is an exciting opportunity for you to solve a realistic problem by applying a complete ML workflow. You will complete this project in teams of 3–4 students. See Section 4 for advice on choosing teammates.

Your task is to build a classifier that solves a 3-class classification problem. You will receive a training dataset consisting of student responses. Your classifier will be evaluated on a separate test set, compiled from responses by TAs and instructors. This test set will not be shared with you, so a major challenge in this project is to build a model that generalizes well to unseen data.

You are encouraged to use any materials from this course, including individual models or ensembles, as well as anything else outside of this course. Ensure that your submission, including the prediction script, stays within the file size limit. Pay attention to avoid underfitting or overfitting and aim for strong, reliable performance on the test set.

Finally, there will be a prize for the team(s) that achieve the best results on the unseen test set. Details will be announced closer to the deadline.

2 How We Used AI to Write This Document

We wrote this document with the assistance of ChatGPT and Claude. By describing how we used AI, we encourage you to be open and to declare how you use AI tools in your own work.

We primarily used AI tools for four purposes: formatting tables, fixing grammatical and spelling mistakes, brainstorming ideas for the two advice sections, and reorganizing the text overall. We read and edited everything these tools produced and kept only what we agreed with. The requirements, the deadlines, and the grading rubrics are our decisions, and we take responsibility for the content.

3 Components and Grading Scheme

The Project is worth 15% of your final course grade. It has the following milestones. The deadline for each milestone is shown on the course syllabus.

Component	Weight
Data Collection	1%
Team Formation	1%
Project Proposal	2%
Prediction Script	3%
Final Report	8%

Table 1: Project Grading Scheme

3.1 Data Collection

The training data for the project will consist entirely of data contributed from students in this course. This means that the quality of the training data depends entirely on the quality of your responses. We highly recommend that you put some thought into your answers.

Please complete the Quercus quiz titled **Project: Data Collection**. This should take 5-10 minutes to complete.

3.2 Team Formation

In the `project_team` assignment on MarkUs, create your group of 3-4 students. Every member must **accept the group invitation** on MarkUs by the deadline to earn the 1% grade. Your team members can be from any section of the course.

Moreover, complete and submit the **group contract** on MarkUs. A group contract template has been posted on Quercus.

3.3 Project Proposal

Submit a PDF file named `proposal.pdf` in the `project_proposal` assignment on MarkUs. The project proposal will be graded based on Table 2.

This milestone exists for two reasons. One, we want to encourage you to start your work early and not wait until the last few weeks of the term. Two, this is an opportunity to receive early feedback and refine your plan for the final report. We encourage you to take the feedback seriously and attend office hours to discuss any questions and concerns with us. Note that receiving full marks for your proposal does **not** mean it is perfect. The TAs had limited time to review your proposal and may not have caught all the issues with it.

Data Exploration

Perform all the work required for the data exploration section and submit a report containing a complete draft of this section. You will also need to submit the code for your work.

Component	Weight
Data Exploration	1%
Planned Methodology	1%

Table 2: Proposal Grading Scheme

Please refer to the project report template for guidelines on what to include in this section. You can also consult the final report grading rubric in table 4 for the important things we will consider in your proposal.

Planned Methodology

Submit a clear draft **plan** for the Methodology section. Your plan should be detailed enough to be **reproducible**, that is, a TA or instructor could implement it independently.

Please refer to the project report template for guidelines for what to include in this section. You can also consult the final report grading rubric in table 4 for the important things that we will look for in your proposal.

3.4 Model Prediction

Submit a Python3 script named `pred.py` to the `project_prediction` assignment on MarkUs.

The script must include a function `predict_all` that takes the name of a CSV file as a parameter and returns predictions for the data in the CSV file.

Requirements

Your submission must meet the following requirements.

- Use Python 3.10 or later.
- `pred.py` may import only: standard Python libraries, `numpy`, and `pandas`. It **cannot** import `sklearn`, PyTorch, or TensorFlow.
- You may submit additional files required by `pred.py`. All submitted files combined must **not** exceed 10MB.
- The script must **not** require networking or download files.
- The script must **run efficiently**: able to produce about 60 predictions within 1 minute using reasonable memory.

Common Confusion: How to Make Predictions Without ML Libraries

A common source of confusion is how to make predictions without importing machine learning libraries. You are free to build and train your model using any tools (e.g., `sklearn`, PyTorch, TensorFlow) and any code you have written or reused from labs. However, your final `pred.py` must generate predictions on the test data without relying on these libraries. One possible approach is to "save" your trained model parameters in a separate file and have `pred.py` load these parameters at runtime to produce predictions.

Model Prediction Grading Criteria

The grading rubric for the prediction script is described in Table 3. The basic criteria include: runnable script, file size at most 10MB, only use allowed imports and uses reasonable memory. For the results over the test set, we will set the thresholds based on the actual prediction accuracies of the student submissions. We leave the threshold choices open so that we can adjust it based on the difficulty of the problem for the semester.

Grade	Quality of the results
3/3	Meets basic criteria. Results on the test set are **outstanding**.
2/3	Meets basic criteria. Results on the test set are **clearly predictive**.
1/3	Meets basic criteria. Results on the test set are **better than random**.
0/3	**Does not** meet basic criteria, or results on the test set are **not better than random**.

Table 3: Model Prediction Grading Rubric

3.5 Report

Submit two files to the `project_report` assignment on MarkUs.

- A PDF file named `report.pdf`.

  This file describes your final model and outlines the steps to develop this model. We have provided a detailed report template in LaTeX. Please comment out the instructions and add your content.

- A ZIP file named `code.zip`.

  This file should contain all the `.py` and/or `.ipynb` files used to develop your final model. Please exclude the `/data` folder from the starter code.

  The `code.zip` file will not be graded but serve as evidence of your work. The files do not need to be runnable by the TA and can rely on external imports (e.g. sklearn) and datasets you don't submit.

The detailed grading rubric for the report is in Table 4.

Section	Marks	Descriptions
Data Exploration (2 marks)	0.5	Describes the feature types, their distributions, and the class balance.
	0.5	Notes missing values, outliers, and inconsistencies, and explains how they were handled.
	0.5	Describes and justifies the preprocessing steps, such as normalization, encoding, and text representation.
	0.5	Explains how the test set was set aside and never used during exploration.
Methodology (4 marks)	0.5	States the three model families used and why they suit this dataset.
	0.5	Describes the optimizer and the training setup, including any regularization or early stopping.
	0.5	Explains the train/validation split or the cross-validation setup.
	1.0	Lists the hyperparameters explored for each model, the ranges and search strategy used, and evidence that the choices were reasonable.
	0.5	Tunes all three models before picking the best one, rather than tuning the winner only.
	1.0	Names the evaluation metrics, includes at least one beyond accuracy, and justifies why they suit this task.
Results (2 marks)	1.0	Reports the metrics from the Methodology section in clear tables or plots, compares the model families, and states the final choice.
	0.5	Identifies the common misclassifications and weaknesses, using a confusion matrix or examples.
	0.5	Gives a single-number estimate of the best model's test performance and justifies it with evidence.
Total	**8.0**	

Table 4: Final Report Grading Rubric

4 How to Choose Your Teammates

The group project is a major part of this course, and who you work with matters as much as how hard you work. A good group makes the project enjoyable, while a bad one can make it stressful. Here is some advice to help you find good teammates.

Look for people whose skills are **different** from yours. This project needs coding, data analysis, writing, and organization, and it is rare for one person to be strong at all of them. If everyone in the group is great at the same thing, some parts of the project will not get the attention they need.

Sort out **schedules and expectations** before you commit. Check that your free hours actually overlap. Agree on how quickly people should reply to messages. If you are someone who starts early, it's a bad idea to collaborate with a person who starts the night before. A mismatch in habits and expectations is more likely to derail a project than a gap in technical skills.

Value **reliability** over raw technical ability. A teammate who shows up, replies to messages, and finishes what they said they would finish is worth more than a brilliant teammate you cannot count on.

Do not limit yourself to **close friends**. Friends can be excellent teammates, but it can also be harder to tell a friend that their work is late or not good enough. Classmates you do not know well can be great to work with, especially they are dependable.

Keep the project deadlines in mind when you choose. There are several milestones spread across the term, so look for teammates who plan ahead and get started without being chased. Someone who is organized and proactive will save your group a lot of stress in the final week.

Finally, be **inclusive**. Some students will not find a group right away, and inviting someone to join costs you nothing. You may gain a teammate with skills and perspectives your group was missing.

In short, a successful group is built on communication, reliability, and respect, not just technical skill. Choose teammates who will make the process worth going through, not only the final grade.

5 How to Work on the Project

Start exploring the data **as soon as it is released**. The groups that struggle are almost always the ones that leave the real work until the last two weeks. By then, there is no time to recover from a model that does not work.

Check in with each other **regularly**. A short weekly meeting is enough to divide up tasks, decide what to try next, and catch problems while they are still small.

Write down what you try **as you go**. Keep a shared document with each experiment you run: which model, which settings, and what validation score you got. Writing the Methodology and Results sections is much easier when you are not trying to remember what you did three weeks ago.

Try **existing models** before building your own. You can compare several model families in sklearn in an afternoon, starting from your lab code. Once you know which one works best on your data, implement that one yourself for `pred.py`.

Leave **time for training**. Some models take a long time to train and tune, so do not plan to run your final experiments the night before a deadline.